

GOLDEN-TRIO

*A trilingual web page translation between
English, Nepali, and Tamang*

Chrome Extension

Google TMT Hackathon 2026

Team Members:

Pratik Pokhrel

Bhisma Bhandari

1. What This Project Is About

Reading content in a language you understand should not require copy-pasting text into a separate tool or opening a new tab. That was the motivation behind building TMT Web Translator. It is a Chrome browser extension that translates any webpage you are reading directly inside the browser, sentence by sentence, without replacing the page or asking you to leave it.

The extension supports all six translation directions between English, Nepali, and Tamang, the three languages the Google TMT API covers. When you turn on translation on a page, the original text nodes in the DOM get replaced in place with their translated equivalents. When you turn it off, every node is restored from a saved copy of the originals, so no reload is ever needed.

The project was built specifically for the Google TMT Hackathon 2026 and uses the hackathon API endpoint as its only translation backend. Everything else, including the DOM walking, rate limiting, translation caching, priority-based rendering, and state restoration, runs entirely inside the browser.

2. Alignment With the Hackathon Theme

The hackathon theme is "Build Tools. Break Barriers. Bridge Languages." This extension addresses all three of those ideas directly.

- Build Tools:
 - The extension is a practical, installable tool that works on any website a user visits, not a demo or a toy. It is designed for real daily use with zero configuration for the end user.
- Break Barriers:
 - Most web content is written in English. Nepali and Tamang speakers who encounter English pages have no seamless way to read them without leaving the page. This tool removes that friction entirely, translating the page in place as you browse.
- Bridge Languages:
 - The TMT API is specifically built around Nepali and Tamang. This extension puts that translation capability directly into the browser where content is consumed, making the bridge as short as possible, one click, no separate interface.

The extension is the only tool in this hackathon context that brings TMT translation directly into the web browsing experience rather than requiring users to navigate to a separate interface.

3. Project Structure

The extension follows a flat structure with a clear separation between the three Chrome extension contexts: the background service worker, the content script, and the popup UI. A scripts folder handles the one build-time step of converting credentials into a loadable module.

File / Folder	Purpose
manifest.json	Extension configuration: permissions, entry points, icons
background.js	Service worker: API calls, translation cache, global rate-limited queue
content.js	Content script: DOM walking, priority translation, node restoration, MutationObserver
content.css	Minimal styles injected with the content script for opacity transitions
config.js	Generated at setup time from .env ,never committed to version control
.env	Your API credentials. Never committed. Listed in .gitignore
.env.example	Template showing which environment variables are required
.gitignore	Excludes .env and config.js from version control
popup/popup.html	Extension popup markup
popup/popup.css	Popup styles with the warm orange-brown design theme
popup/popup.js	Popup logic: language selection, start/stop, progress bar, chip navigation
scripts/generate-config.mjs	Node.js script: reads .env and writes config.js for the service worker to import
icons/	Extension icons at 16px, 48px, and 128px (all using icon.png)

4. Setup and Installation

Setup takes about five minutes. You need Node.js 18 or newer installed, and your API token.

Step 1: Get the project

Extract or clone the project folder to any location on your machine.

Step 2: Create your credentials file

```
cp .env.example .env
```

Open .env in any text editor and fill in the two values:

```
TMT_API_KEY=YOUR_TEAM_TOKEN  
TMT_API_ENDPOINT=YOUR_API_ENDPOINT
```

Step 3: Generate config.js

This script reads your .env file and writes a config.js file that the extension imports at runtime:

```
node scripts/generate-config.mjs
```

Why this step? Chrome extensions cannot read .env files directly. The generator converts your credentials into a JavaScript module that the service worker can import. Both .env and config.js are in .gitignore so neither file ever gets committed to a repository.

Step 4: Load into Chrome

- Open Chrome and navigate to chrome://extensions
- Turn on Developer mode using the toggle in the top-right corner
- Click Load unpacked and select the project folder
- The TMT icon will appear in the Chrome toolbar ,you are ready to translate

5. How It Works

The extension is split across three isolated execution environments that communicate through Chrome's message passing system. Each part has a single, well-defined responsibility.

Context	File	What it does
Popup	popup.js	Shows the UI, lets the user pick languages, manages start/stop, displays a live progress bar, and persists preferences per session
Service Worker	background.js	Handles all API calls, enforces the rate limit through a global queue, maintains a session-scoped translation cache, and handles 429 backoff with retry
Content Script	content.js	Walks the DOM, prioritises main content, sends text to the service worker, writes translations back, tracks progress, and handles page restoration on toggle-off

5.1 The Translation Flow

Here is what happens from the moment you click Start Translation:

- Toggle ON:
 - The popup sends a startTranslation message to the content script running on the current tab, passing the chosen source and target language codes.
- DOM Walk:
 - The content script creates a TreeWalker that visits every text node in the document body. It skips script tags, style blocks, editable regions, iframes, SVG, canvas, and any node already present in the originalTexts map.
- Priority Sort:
 - Nodes are split into three buckets: high priority (inside main, article, or [role='main']), medium priority (everything else), and low priority (nav, header, footer, aside). High and medium nodes are translated first; peripheral UI is deferred.
- Queue and rate limit:
 - Each text node is sent as a translation message to the background service worker, which places it in a global queue draining at one request per second, keeping total requests comfortably below the 60/min API ceiling.
- Cache check:
 - Before every API call the service worker checks an in-memory Map. If a matching translation already exists for the same text and language pair it is returned immediately with no API request made.
- API call:
 - The service worker sends a POST request to the TMT endpoint with the text and language codes. The response carries the translated string in the output field.
- DOM update:
 - The translated text is written back to the node's nodeValue. Leading and trailing whitespace from the original is preserved. The node reference is stored in an originalTexts Map for later restoration.
- Progress reporting:
 - After each node is written, the content script calculates completion percentage and fires a chrome.runtime.sendMessage to the popup, which updates the live progress bar.
- Dynamic content:
 - A MutationObserver with a 120ms debounce watches for new nodes being added to the DOM (common on social media feeds, SPAs, and infinite scroll pages) and queues them automatically.
- Toggle OFF:
 - The content script iterates through the originalTexts Map and restores each node in place. No reload happens. The observer is disconnected. The page returns to its original language instantly.

6. Code Architecture

The architecture was shaped around two constraints that emerged during development: Chrome's message channel timeout, and the TMT API rate limit. Every significant design decision traces back to one of those two constraints.

6.1 Sequential Node Translation

Earlier prototypes sent all nodes in a single `sendMessage()` call and waited for a single response containing all translations. The background service worker would queue all requests at one per second, meaning the response could take over a minute. Chrome closes message channels that receive no response within roughly five seconds. The result was that callbacks were silently dropped and nothing appeared on the page.

The current approach sends one node at a time and `awaits` each response before sending the next. The sequential loop is slower in theory but it is the only approach that works reliably within Chrome's constraints. The progress bar mitigates the perception of slowness by showing real-time advancement.

6.2 In-Place Restoration via `originalTexts` Map

Before each node's `nodeValue` is overwritten, the content script stores the original string in a `Map<Node, string>` called `originalTexts`. When translation is toggled off, the map is iterated and each node is restored from its saved value. An `isConnected` check prevents errors if a node was removed from the DOM while translation was running.

This approach means users never lose dynamically loaded content when turning translation off. A page with a live feed or chat that was translated retains all the messages that appeared since page load.

6.3 Global Rate-Limited Queue

The service worker maintains a single `requestQueue` array and a `drainQueue()` loop. Every translation request is wrapped in a Promise and pushed into this queue. The drain loop enforces a minimum gap of 1000ms between each API call (approximately 54 requests per minute), and respects a global `backoffUntil` timestamp when a 429 response is received.

6.4 Session-Scoped Translation Cache

Translated strings are stored in an in-memory `Map` keyed by `srcLang:tgtLang:text`. This cache is also persisted to `chrome.storage.session` on a 350ms debounced timer, so it survives service worker restarts within the same browser session. On startup, the cache is hydrated from session storage. The cache is capped at 1000 entries (LRU eviction of the oldest) to avoid unbounded memory growth.

The content script also maintains a local in-memory cache (capped at 1500 entries) so that nodes with identical text on the same page do not even send a message to the background, they resolve instantly from memory.

6.5 In-Flight Deduplication

The content script tracks pending translation promises in an `inFlightTranslations Map`. If two text nodes on the page contain the same string and are sent for translation before the first response arrives, the second call receives the same Promise as the first rather than launching a duplicate request. Once the promise resolves, the entry is removed from the in-flight map.

6.6 Priority-Based Translation Order

Rather than translating all nodes in document order, the content script separates them into three priority groups:

Priority	Selector	Effect
0 : High	main, article, [role='main']	Translated first. The main reading content appears in the target language immediately.
1 : Medium	Everything else	Body content not inside a semantic landmark. Translated in the same first wave as high-priority.
2 : Low	header, footer, aside	Deferred to a background pass via <code>setTimeout</code> so it does not block the reading experience.

6.7 MutationObserver for Dynamic Content

After the initial translation pass, a `MutationObserver` is started with `childList: true, subtree: true`. New text nodes and new element nodes (from which text children are extracted) are collected into a pending Set. A 120ms debounce timer fires once per burst of mutations, translating all newly arrived nodes in one enqueue call. The observer is disconnected when translation is stopped.

6.8 Duplicate Script Guard

Because the popup may programmatically inject `content.js` into tabs that were open before the extension was loaded, the script begins by checking `globalThis.__TMT_CONTENT_SCRIPT_LOADED__`. If it is already set, the script exits immediately without re-registering any listeners or state. This prevents double-registration of the message listener and duplicate translation runs.

6.9 Extension Context Invalidation Handling

When the extension is reloaded or updated while a page is being translated, `chrome.runtime.sendMessage()` throws `'Extension context invalidated'`. The content script catches this both at the synchronous level with a try-catch and at the callback level by checking `chrome.runtime.lastError`. When caught, the translation loop halts cleanly with no uncaught exceptions appearing in the console.

6.10 Run ID Versioning

Every call to start or stop translation increments a `translationRunId` integer. Every async operation (batches, queue entries, observer callbacks) carries the run ID from when it was created and checks it before doing any work. If the ID no longer matches the current

`translationRunId`, the operation aborts silently. This eliminates race conditions when the user rapidly toggles translation or switches language pairs.

7. Rate Limiting Strategy

The TMT API allows 60 requests per minute per token. The extension uses three layers to stay comfortably under that limit.

Layer	How it works
Minimum gap	A 1000ms minimum is enforced between every API call inside the global queue. At most 54 requests can go out per minute, leaving a comfortable buffer below the 60/min ceiling.
Session cache	Translations already fetched in the current session are returned from the in-memory or session-storage cache with zero API calls. On pages with repeated phrases this significantly reduces actual request volume.
Content script cache	Identical text nodes on the same page resolve from the local content-script cache without even reaching the background service worker, further reducing queue pressure.
429 backoff with retry	If the API returns a 429, the Retry-After header is read. Both numeric-seconds and HTTP-date formats are supported. The queue sleeps for that duration (or REQUEST_GAP_MS if no header is present) before retrying the failed request automatically.

8. Permissions Explained

The extension requests only what it genuinely needs. Every permission has a specific reason and none are requested for future use.

Permission	Why it is needed
activeTab	Lets the popup query the current tab's ID so it can send startTranslation and stopTranslation messages to the correct content script instance.
scripting	Allows the popup to programmatically inject content.js into tabs that were already open before the extension was loaded. Without this, the toggle would silently fail on any pre-existing tab.
storage	Saves the user's chosen language pair to chrome.storage.local so preferences survive popup closes and browser restarts. Also used by the service worker to persist the translation cache to chrome.storage.session.
host_permissions: <all_urls>	Required for the content script to run on any website and for the service worker's fetch() to call the TMT API endpoint regardless of the page origin.

9. User Guide

9.1 Translating a Page

- Click the TMT extension icon in the Chrome toolbar to open the popup
- Select your source language and target language from the dropdowns, or click one of the six direction chips (EN→NE, EN→TM, NE→TM, and their reverses)
- The swap button between the dropdowns reverses the direction instantly
- Click Start Translation. The progress bar appears and the page begins updating node by node
- Click Stop at any time to restore the original text in place. No page reload happens

9.2 Progress Bar

While translation is running, the popup shows a live progress bar with a percentage counter. This reflects the proportion of text nodes on the page that have been translated. The counter updates at most once every 120ms to avoid flooding the message channel. It reaches 100% when all nodes discovered at translation start (plus any subsequently observed via MutationObserver) have been processed.

9.3 Dynamic Pages

Pages that load content as you scroll, or single-page applications that update the DOM without a full navigation, are handled by a MutationObserver in the content script. Newly added text nodes are picked up within 120ms of appearing and queued for translation automatically. This covers social media feeds, infinite scroll news sites, and chat applications.

9.4 Language Persistence

Your last-used source and target language pair is saved to Chrome's local storage and restored the next time you open the popup. The active chip is also updated to match.

9.5 What Does Not Work

- PDFs in Chrome's built-in viewer:
 - The viewer runs inside a sandboxed extension page that blocks all external content scripts. This is a Chrome security boundary with no available workaround in Manifest V3.
- Very long pages:
 - Because of the one-request-per-second rate limit, a 500-node page takes roughly eight to nine minutes to fully translate. The progress bar makes this transparent to the user.
- Cross-origin iframes:
 - Chrome blocks content scripts from accessing iframes served from a different origin, even with `all_frames: true` in the manifest. Content inside cross-origin iframes is not translated.
- Pages that block content scripts:
 - Some pages (`chrome://`, the Chrome Web Store, and other extension pages) have security policies that prevent any extension content script from running. The popup catches injection errors and displays 'Can't inject on this page'.

10. Security and Credential Handling

The API token is never written directly into any source file. The flow keeps credentials local and out of version control at every step.

- `.env.example` is the only credential-related file in version control. It contains placeholder variable names and nothing else.
- `.env` holds your real API token and endpoint. It is in `.gitignore` and must never be committed.
- `config.js` is also in `.gitignore`. Even though it does not contain the raw `.env` text, it contains the derived values and is treated as equally sensitive.
- The `generate-config.mjs` script reads `.env` at build time and writes `config.js`. If credentials need to be rotated, update `.env` and re-run the script, no source file ever needs editing.

⚠ If you ever push this project to a public repository, double-check that `.env` and `config.js` are absent from the commit history. Git does not automatically remove files from history just because they are in `.gitignore`. Use `git log --all -- config.js` to verify.

11. Deployment and Long-Term Maintenance

11.1 Updating the API Key

Edit `.env` with the new token and run the generator script again:

```
node scripts/generate-config.mjs
```

Then reload the extension at `chrome://extensions`. The new key takes effect immediately.

11.2 Changing the Rate Limit

The gap is controlled by a single constant at the top of `background.js`:

```
const REQUEST_GAP_MS = 1000; // 1 second ≈ 54 requests per minute
```

Raise this value to be more conservative, or lower it if the API limit is increased.

11.3 Adding a New Language

Three things need to change across the codebase:

- In `popup.html`:
 - Add a new `<option>` element to both the Source and Target `<select>` dropdowns using the language name as display text and the exact API language code as the value.

- In popup.html:
 - Add corresponding direction chip <button> elements inside .direction-chips for the new language paired with each existing language in both directions.
- In popup.js:
 - Verify that the chip's data-src and data-tgt attributes exactly match the value strings used in the dropdowns and accepted by the API. The TMT API accepts full names, two-letter codes, and three-letter abbreviations, confirming which format applies.

11.4 Upgrading to a Future Manifest Version

The extension uses Manifest V3, the current stable standard for Chrome. If a Manifest V4 ever becomes available, the main migration points would be the service worker lifecycle model and the scripting permission API, both of which are already in their MV3 form here. No changes to content.js or the popup would be expected.

12. Known Limitations and Suggested Improvements

The following areas represent the most impactful improvements that could be made to the extension. They are listed in order of the benefit they would deliver.

Area	What could be done
Speed on large pages	The one-request-per-second rate means a 500-node page takes roughly eight minutes. If the TMT API ever supports batch translation in a single request, the rate of translation could increase dramatically. The batchNodes() function in content.js is already structured to support this, it groups nodes into batches that could be sent as a single payload.
Popup closes, progress is lost	Progress messages from content.js are only received while the popup is open. If the user closes the popup during translation and reopens it, the progress bar resets to 0%. The fix is to have the content script write progress to chrome.storage.session and have the popup read it on open.
No persistent translation state across navigations	If a user navigates away from a translated page and then hits the back button, the content script restarts in a fresh state. Previously cached translations in the service worker will still be used, but the visual translation state is not restored.
PDF viewer incompatibility	A dedicated PDF upload flow using pdf.js could extract text from a PDF file selected by the user, translate it via the TMT API, and render the result without needing to touch the Chrome PDF viewer DOM.
No error count visible to user	When the API returns a failure for a node, the extension silently falls back to the original text. Exposing a count of failed nodes in the popup (e.g. '3 nodes could not be translated') would help users understand partial translation results.